

Sistemas Operativos

Guía Práctica N° 7

Disco y Sistema de Archivos

Resolución completa explicada paso a paso
Basada en la clave de respuestas del profesor

Índice

Índice	2
Cómo está organizado este documento	3
Bloque 1 — Geometría y capacidad del disco	4
Ejercicio 1 — Capacidad a partir de la geometría	4
Ejercicio 2 — Capacidad máxima según los bits de direccionamiento	4
Ejercicio 3 — Capacidad, bloques y direccionamiento	4
Bloque 2 — Sistema de archivos FAT	6
Ejercicio 4 — FAT16 con bloques de 4 KB	6
Ejercicio 5 — Sectores que ocupa la FAT (FAT32, 20 GB)	6
Ejercicio 6 — Sectores mínimos por clúster	7
Ejercicio 7 — FAT16, máximo direccionable y bloque mínimo	7
Ejercicio 8 — Recorrido de la lista enlazada en la FAT	7
Ejercicio 9 — Reconstruir el archivo desde EOF	8
Bloque 3 — Sistemas de archivos basados en i-nodos	9
Ejercicio 10 — Bloques de enlaces para 1548 KB	9
Ejercicio 11 — Máximos en un i-nodo (bloque 1 KB)	9
Ejercicio 12 — Bloques de enlaces (bloque 4 KB)	10
Ejercicio 13 — Causas que limitan el tamaño de un archivo	10
Bloque 4 — Asignación de espacio en disco	12
Ejercicio 14 — Encadenamiento vs. indexación	12
Bloque 5 — Planificación del brazo del disco	13
Ejercicio 15 — Cálculo del recorrido por cada algoritmo	13
Ejercicio 16 — Llega el pedido 47 mientras el brazo busca el 45.	15

Cómo está organizado este documento

La guía se agrupa en cinco bloques temáticos. Cada ejercicio incluye el enunciado, los conceptos clave que necesitas dominar, la resolución paso a paso (con el porqué de cada cuenta) y la respuesta final tal como la espera tu profesor.

- **Bloque 1 — Geometría y capacidad del disco** (ej. 1 a 3): cilindros, cabezas, sectores y cómo se calcula la capacidad y el direccionamiento de bloques.
- **Bloque 2 — Sistema de archivos FAT** (ej. 4 a 9): tabla FAT, entradas, clústeres, FAT16/FAT32 y recorrido de la lista enlazada.
- **Bloque 3 — Sistemas basados en i-nodos** (ej. 10 a 13): índices directos, indirectos simples, dobles y triples; bloques de datos vs. bloques de enlaces.
- **Bloque 4 — Asignación de espacio** (ej. 14): encadenamiento vs. indexación y la pérdida de espacio.
- **Bloque 5 — Planificación del brazo del disco** (ej. 15 y 16): FIFO, SSTF, SCAN, LOOK, C-SCAN y C-LOOK.

Atención: La clave de tu profesor tiene **erratas en algunos totales del Bloque 5** (y un número del ej. 12). Las dejo marcadas con el valor de la clave y el valor correcto según el propio método, para que no te confundan en el examen. Conviene confirmar la convención exacta con tu profesor.

Bloque 1 — Geometría y capacidad del disco

Conceptos clave

Un disco rígido se organiza físicamente en **platos**. Cada plato tiene dos **caras** y cada cara es leída por una **cabeza** de lectura/escritura. Cada cara se divide en **pistas** circulares concéntricas, y cada pista en **sectores** (la unidad física mínima de lectura/escritura, típicamente 512 bytes).

El conjunto de pistas que quedan alineadas verticalmente (la misma pista en todas las caras) forma un **cilindro**. Por eso: **Nº de cilindros = Nº de pistas por cara**, y **Nº de cabezas = Nº de caras**.

La fórmula maestra de capacidad es:

| Capacidad = Nº cilindros × Nº cabezas × Nº sectores/pista × bytes/sector

El **sistema operativo** no trabaja con sectores sino con **bloques** (agrupaciones de varios sectores). Un bloque es la unidad lógica de asignación. Para direccionar B bloques se necesitan $\lceil \log_2 B \rceil$ bits.

Ejercicio 1 — Capacidad a partir de la geometría

Enunciado. Disco con 16 cabezas, 63 sectores/pista y 6253 cilindros. ¿Capacidad de almacenamiento? (sector = 512 bytes).

Resolución

Aplicamos directamente la fórmula maestra. Conviene calcularlo por etapas para entender qué representa cada factor:

| Capacidad de una pista = 512 B/sector × 63 sectores = 32.256 B/pista
 | Capacidad de una cara = 32.256 B/pista × 6.253 pistas = 201.696.768 B/cara
 | Capacidad del disco = 201.696.768 B × 16 caras = 3.227.148.288 B

Convertimos a unidades mayores dividiendo por 1024 sucesivamente:

| 3.227.148.288 B ≈ 3.151.512 KB ≈ 3.077,6 MB ≈ 3 GB

| **Respuesta:** La capacidad del disco es de **3.227.148.288 bytes ≈ 3 GB**.

Ejercicio 2 — Capacidad máxima según los bits de direccionamiento

Enunciado. El disco usa 16 bits para el cilindro, 5 bits para la cabeza y 10 bits para el sector (sectores de 512 bytes). ¿Capacidad máxima?

Resolución

La cantidad de bits de cada campo determina **cuántos valores distintos** se pueden direccionar: con k bits se direccionan 2^k elementos. Traducimos cada campo:

| Cilindros = 2^{16} = 65.536 Cabezas = 2^5 = 32 Sectores/pista = 2^{10} = 1.024

Aplicamos la fórmula maestra dejando todo como potencias de 2 ($512 = 2^9$):

| Capacidad = $2^{16} \times 2^5 \times 2^{10} \times 2^9 = 2^{(16+5+10+9)} = 2^{40} = 1 \text{ TB}$

| **Respuesta:** La capacidad máxima del disco es **2^{40} bytes = 1 TB**.

Ejercicio 3 — Capacidad, bloques y direccionamiento

Enunciado. Disco con 8 cabezas, 512 pistas, 128 sectores/pista y 256 bytes/sector. El controlador lee una pista completa por operación. El SO usa bloques de 4 KB.

Pasamos los datos a potencias de 2: 8 cabezas = 2^3 , 512 pistas = 2^9 , 128 sectores = 2^7 , 256 B = 2^8 .

a) Capacidad total del disco

$$| \text{Capacidad} = 512 \times 8 \times 128 \times 256 = 2^9 \times 2^3 \times 2^7 \times 2^8 = 2^{27} = 128 \text{ MB}$$

Respuesta: 128 MB (2^{27} bytes).

b) Sectores que ocupa un bloque

Dividimos el tamaño del bloque entre el tamaño del sector:

$$| N_s = \text{tamaño bloque} / \text{tamaño sector} = 4 \text{ KB} / 256 \text{ B} = 2^{12} / 2^8 = 2^4 = 16 \text{ sectores}$$

Respuesta: Un bloque ocupa 16 sectores.

c) Bloques que lee el controlador por operación

El controlador lee una pista completa (128 sectores). Como cada bloque son 16 sectores:

$$| \text{Bloques/pista} = 128 \text{ sectores} / 16 \text{ sectores/bloque} = 8 \text{ bloques}$$

Respuesta: El controlador lee 8 bloques en cada operación.

d) Bits para direccionar un bloque

Primero el número total de bloques del disco, y de ahí los bits ($2^{\text{bits}} = N^{\circ} \text{ bloques}$):

$$| N^{\circ} \text{ bloques} = \text{capacidad} / \text{tamaño bloque} = 2^{27} \text{ B} / 2^{12} \text{ B} = 2^{15} \text{ bloques} \rightarrow 15 \text{ bits}$$

Respuesta: Se necesitan 15 bits.

e) Bloques que contiene un cilindro

Un cilindro tiene 8 pistas (una por cabeza). Calculamos su capacidad y la dividimos por el tamaño de bloque:

$$| \begin{aligned} \text{Cap. cilindro} &= 8 \times 128 \times 256 = 2^3 \times 2^7 \times 2^8 = 2^{18} \text{ B} \\ \text{Bloques/cilindro} &= 2^{18} \text{ B} / 2^{12} \text{ B} = 2^6 = 64 \text{ bloques} \end{aligned}$$

Respuesta: Un cilindro contiene 64 bloques.

Bloque 2 — Sistema de archivos FAT

Conceptos clave

En **FAT** (File Allocation Table) el disco se divide en **clústeres** (también llamados bloques o agrupamientos), formados por un número fijo de sectores. La **tabla FAT** tiene **una entrada por cada clúster** del disco.

Los clústeres de un archivo se enlazan como una **lista enlazada**: la entrada de la FAT de un clúster guarda el número del **siguiente** clúster del archivo; el último clúster contiene la marca **EOF** (fin de lista).

El número en el nombre indica los **bits por entrada**: FAT16 → entradas de 16 bits → $2^{16} = 65.536$ clústeres direccionables; FAT32 → entradas de 32 bits. Dos fórmulas que se usan todo el tiempo:

Capacidad máx. direccionable = N° entradas × tamaño de clúster
Tamaño de la tabla FAT = N° entradas × tamaño de cada entrada

Ejercicio 4 — FAT16 con bloques de 4 KB

Enunciado. Partición FAT16 formateada con bloques de 4 KB.

a) Máximo direccionable teóricamente

FAT16 tiene 2^{16} entradas; cada entrada apunta a un bloque de 4 KB (2^{12} B):

Máximo = 2^{16} entradas × 2^{12} B = 2^{28} = 256 MB

Respuesta: 256 MB.

b) ¿Es adecuado para una partición de 8 GB?

No. Con bloques de 4 KB sólo se pueden direccionar 256 MB, así que de una partición de 8 GB **quedarían sin usar** casi todos los 8 GB (sólo se aprovecharían 256 MB).

Respuesta: No es adecuado: con bloque de 4 KB sólo se direccionan 256 MB de los 8 GB.

c) Tamaño de bloque adecuado y espacio que ocupa la FAT

El bloque mínimo se obtiene repartiendo la partición entre las 2^{16} entradas disponibles:

Tamaño bloque = $8 \text{ GB} / 2^{16} = 2^{33} / 2^{16} = 2^{17} = 128 \text{ KB}$ por bloque

Tamaño de la tabla FAT = entradas × tamaño de entrada:

FAT = 2^{16} entradas × 16 bits = $2^{16} \times 2 \text{ B} = 2^{17} = 128 \text{ KB}$

Respuesta: El bloque debe ser de **128 KB**; la FAT ocupa **128 KB**, es decir **exactamente 1 bloque**.

Ejercicio 5 — Sectores que ocupa la FAT (FAT32, 20 GB)

Enunciado. Disco de 20 GB, una partición FAT32, clúster = 16 sectores × 512 B. ¿Cuántos sectores ocupa cada copia de la FAT?

Resolución

Paso 1 — N° de entradas. Hay una entrada por clúster; el clúster mide $16 \times 512 \text{ B} = 8 \text{ KB}$:

N° entradas = $20 \text{ GB} / (16 \times 512 \text{ B}) = 20 \times 2^{30} / 2^{13} = 20 \times 2^{17} = 2.621.440$ entradas

Paso 2 — Tamaño de la FAT. Cada entrada FAT32 mide 32 bits = 4 B:

$$\text{Tamaño FAT} = 20 \times 2^{17} \text{ entradas} \times 32 \text{ bits} = 20 \times 2^{17} \times 4 \text{ B} = 20 \times 2^{19} \text{ B}$$

Paso 3 — Clústeres que ocupa la FAT, dividiendo por el tamaño del clúster:

$$\text{Clústeres} = (20 \times 2^{19}) / (16 \times 512) = 20 \times 2^{19} / 2^{13} = 20 \times 2^6 = 1.280 \text{ clústeres}$$

Paso 4 — Sectores, multiplicando por 16 sectores/clúster:

$$\text{Sectores} = 1.280 \times 16 = 20.480 \text{ sectores}$$

Respuesta: Cada copia de la FAT ocupa **20.480 sectores**.

Ejercicio 6 — Sectores mínimos por clúster

Enunciado. Archivo de 160 KB que usa 10 entradas de la FAT; sector = 512 B. ¿Cuántos sectores como mínimo forman cada clúster?

Resolución

El archivo, en sectores, ocupa:

$$\text{Sectores del archivo} = 160 \text{ KB} / 512 \text{ B} = 163.840 / 512 = 320 \text{ sectores}$$

Usa 10 entradas = 10 clústeres. Repartiendo los 320 sectores entre esos 10 clústeres:

$$\text{Sectores por clúster} = 320 \text{ sectores} / 10 \text{ clústeres} = 32 \text{ sectores/clúster}$$

Es el **mínimo** porque suponemos que el archivo llena por completo sus clústeres; si el último estuviera parcialmente lleno, el clúster sería aún mayor.

Respuesta: Cada clúster está formado por **32 sectores como mínimo**.

Ejercicio 7 — FAT16, máximo direccionable y bloque mínimo

Enunciado. Partición FAT16. (i) Con bloque de 1 KB, ¿cuántos KB se direccionan como máximo? (ii) Si la partición es de 2 GB, ¿qué tamaño mínimo de bloque hace falta?

(i) Máximo con bloques de 1 KB

$$\text{Máximo} = 2^{16} \text{ bloques} \times 1 \text{ KB} = 65.536 \text{ KB} = 64 \text{ MB}$$

Respuesta: 64 MB.

(ii) Bloque mínimo para 2 GB

Repartimos la partición entre las 2^{16} entradas:

$$\text{Bloque mín.} = 2 \text{ GB} / 2^{16} = 2^{31} \text{ B} / 2^{16} = 2^{15} = 32.768 \text{ B} = 32 \text{ KB}$$

Respuesta: El bloque debe ser de **32 KB como mínimo**.

Ejercicio 8 — Recorrido de la lista enlazada en la FAT

Enunciado. Contenido de clústeres y enlaces de la FAT; el archivo empieza en el clúster 1212.

Clúster	1210	1211	1212	1213	1214	1215	1216	1217	1218	1219	1220	1221	1222
Contenido	T	J	A	M	R	W	X	Z	U	S	B	P	K
Siguiente	EOF	1214	1213	1219	EOF	1216	1217	EOF	1222	1218	1221	2110	1211

a) Información del archivo

Partiendo del clúster inicial (1212), seguimos la columna **Siguiente** y vamos anotando el **Contenido** de cada clúster hasta llegar a EOF:

1212(A) → 1213(M) → 1219(S) → 1218(U) → 1222(K) → 1211(J) → 1214(R) → EOF

Respuesta: La información del archivo es **A M S U K J R** (ocupa **7 clústeres**). No se puede saber el tamaño exacto en bytes sin conocer cuántos sectores tiene cada clúster.

b) Clústeres máximos referenciables (FAT16)

Máximo = $2^{16} = 65.536$ clústeres

Respuesta: 65.536 clústeres.

c) Tamaño de clúster y sectores (disco de 2 GB)

Tamaño clúster = $2^{31} \text{ B} / 2^{16} = 2^{15} = 32.768 \text{ B} = 32 \text{ KB}$

Sectores/clúster = $32.768 / 512 = 64$ sectores

Respuesta: Cada clúster mide **32 KB** y tiene **64 sectores**.

d) Tamaño del archivo en disco

El archivo ocupa 7 clústeres completos:

Tamaño = $32.768 \text{ B/clúster} \times 7 = 229.376 \text{ B} = 224 \text{ KB}$

Respuesta: El archivo ocupa **224 KB** en disco.

Ejercicio 9 — Reconstruir el archivo desde EOF

Enunciado. Reconstruir el contenido del archivo a partir de los enlaces de la FAT.

Clúster	10	11	12	13	14	15
Contenido	H	S	Y	N	S	B
Siguiente	11	EOF	13	10	15	12

Resolución

Acá no nos dan el clúster inicial, pero sí sabemos que la lista termina en **EOF** (entrada 11). El truco: el **clúster inicial es el único que no aparece como 'siguiente' de ningún otro**.

Revisando, a la entrada 14 nadie apunta, así que el archivo empieza en 14. Recorremos siguiendo los enlaces:

14 → 15 → 12 → 13 → 10 → 11 → EOF

Y leemos el contenido de cada uno:

S → B → Y → N → H → S → EOF

Respuesta: El contenido del archivo es **S B Y N H S**.

Bloque 3 — Sistemas de archivos basados en i-nodos

Conceptos clave

Cada archivo tiene un **i-nodo** con un conjunto de punteros a bloques. Hay dos tipos de bloques: los **bloques de datos** (contienen el contenido del archivo) y los **bloques de enlaces/índices** (contienen punteros a otros bloques). Tipos de punteros del i-nodo:

- **Directo:** apunta a 1 bloque de datos. No gasta bloques de índices.
- **Indirecto simple:** apunta a 1 bloque de índices que, a su vez, apunta a k^* bloques de datos (donde $k^* = \text{índices por bloque}$). Gasta **1** bloque de enlaces.
- **Indirecto doble:** apunta a 1 bloque de índices que apunta a k^* bloques de índices, cada uno con k^* datos $\rightarrow k^{2*}$ datos. Gasta **$1 + k$** bloques de enlaces (1 de primer nivel + k de segundo nivel).
- **Indirecto triple:** llega a k^{3*} datos; gasta **$1 + k + k^2$** bloques de enlaces.

El número de índices por bloque es la magnitud central:

$$k = \text{índices por bloque} = \text{tamaño del bloque} / \text{tamaño del puntero (índice)}$$

Ejercicio 10 — Bloques de enlaces para 1548 KB

Enunciado. i-nodo con 5 directos, 3 indirectos simples, 2 indirectos dobles y 1 indirecto triple. Bloque = 2 KB, puntero = 64 bits. Archivo de 1548 KB. ¿Cuántos bloques almacenan enlaces?

Resolución

Paso 1 — Bloques de datos del archivo:

$$1.548 \text{ KB} / 2 \text{ KB} = 774 \text{ bloques de datos}$$

Paso 2 — Índices por bloque (puntero 64 bits = 8 B):

$$k = 2 \text{ KB} / 8 \text{ B} = 2.048 / 8 = 256 \text{ índices por bloque}$$

Paso 3 — Vamos cubriendo los 774 datos en orden, gastando los enlaces mínimos:

- Los **5 directos** cubren 5 datos y gastan **0** bloques de enlaces. Faltan $774 - 5 = 769$.
- Cada **indirecto simple** cubre 256 datos y gasta 1 enlace. Con los **3** simples cubrimos $3 \times 256 = 768$ datos y gastamos **3** enlaces. Faltan $769 - 768 = 1$ dato.
- Para ese **1 dato** restante usamos el **primer indirecto doble**, que necesita lo mínimo: **2 bloques de enlaces** (1 de primer nivel + 1 de segundo nivel) para poder llegar a 1 bloque de datos.

$$\text{Total de bloques de enlaces} = 3 \text{ (simples)} + 2 \text{ (doble)} = 5$$

Respuesta: Se utilizan **5 bloques** para almacenar enlaces.

Ejercicio 11 — Máximos en un i-nodo (bloque 1 KB)

Enunciado. Bloque = 1 KB, puntero = 32 bits. i-nodo con 2 directos, 2 indirectos simples y 1 indirecto doble.

Índices por bloque (puntero 32 bits = 4 B):

$$k = 1 \text{ KB} / 4 \text{ B} = 2^{10} / 2^2 = 2^8 = 256 \text{ índices por bloque}$$

a) Bloques de enlaces si el archivo es lo más grande posible

- Directos: 0 bloques de enlaces.

- Cada indirecto simple: 1 bloque de enlaces $\rightarrow$ 2 simples = **2**.
- Indirecto doble: $1 + k = 1 + 256 = \mathbf{257}$.

| Total enlaces = $2 + (1 + 256) = 259$ bloques de índices

Respuesta: 259 bloques de enlaces.

b) Bloques de datos máximos

- 2 directos $\rightarrow$ 2 datos.
- 2 indirectos simples $\rightarrow 2 \times 256 = 512$ datos.
- 1 indirecto doble $\rightarrow 256^2 = 65.536$ datos.

| Total datos = $2 + 512 + 65.536 = 66.050$ bloques de datos

Respuesta: 66.050 bloques de datos.

c) Tamaño máximo del archivo

| Tamaño máx. = $66.050 \text{ bloques} \times 1 \text{ KB} = 66.050 \text{ KB}$

Respuesta: 66.050 KB.

d) Tamaño que ocuparía en disco

En disco se cuentan **datos + enlaces**:

| $(66.050 + 259) \text{ bloques} \times 1 \text{ KB} = 66.309 \text{ KB}$

Respuesta: Ocupa 66.309 KB en disco.

Ejercicio 12 — Bloques de enlaces (bloque 4 KB)

Enunciado. Bloque = 4 KB, puntero = 32 bits. i-nodo con 2 directos, 2 indirectos simples y 1 indirecto doble. ¿Cuántos bloques contendrán enlaces si el archivo es máximo?

Resolución

| $k = 4 \text{ KB} / 32 \text{ bits} = 4 \text{ KB} / 4 \text{ B} = 1.024$ índices por bloque

- Directos: 0 enlaces.
- 2 indirectos simples: 1 cada uno $\rightarrow$ 2 enlaces.
- Indirecto doble: $1 + 1.024 = 1.025$ enlaces.

| Total enlaces = $2 + (1 + 1.024) = 1.027$ bloques de índices

Respuesta: 1.027 bloques contienen enlaces.

Como dato adicional, la clave también calcula los bloques de **datos** máximos: $2 + 2 \cdot 1.024 + 1.024^2 = 2 + 2.048 + 1.048.576$.

Atención: La clave imprime ese total de **datos** como 1.051.626, pero la suma correcta es **1.050.626** ($2 + 2.048 + 1.048.576$). Es una errata; la pregunta del ejercicio sólo pide los **1.027** bloques de enlaces, que están bien.

Ejercicio 13 — Causas que limitan el tamaño de un archivo

Enunciado. Dispositivo con bloques de 2 KB, direcciones de 64 bits. El descriptor tiene: 1 B tipo, 2 B propietario, 4 B tamaño, 1 puntero directo, 1 indirecto simple y 1 indirecto doble. Indicar las causas que limitan el tamaño de un archivo y el máximo.

Causa 1 — El campo 'tamaño' del descriptor

El tamaño se guarda en **4 bytes = 32 bits**, así que el mayor valor representable es:

Máx. por el campo tamaño = 2^{32} bytes = 4 GB

Causa 2 — La estructura de punteros del descriptor

El máximo de bloques accesibles depende de los punteros (puntero 64 bits = 8 B, $k = 2 \text{ KB} / 8 \text{ B} = 256$):

- 1 directo $\rightarrow$ 1 bloque.
- 1 indirecto simple $\rightarrow k = 256$ bloques.
- 1 indirecto doble $\rightarrow 256 \times 256 = 65.536$ bloques.

Máx. en bloques = $1 + 256 + 65.536 = 65.793$ bloques

Máx. por punteros = $65.793 \times 2 \text{ KB} = 131.586 \text{ KB} \approx 128,5 \text{ MB}$

Respuesta: El tamaño máximo real es el **menor** de los dos límites. Como $131.586 \text{ KB} (\approx 128,5 \text{ MB}) < 4 \text{ GB}$, **la estructura de punteros es la que limita**: el archivo no puede superar los **131.586 KB**, aunque el campo tamaño permitiría hasta 4 GB.

Direcccionamiento del dispositivo (dato extra de la clave)

Con punteros de 64 bits se direccionan 2^{64} bloques. El mapa de bits de bloques (N bloques) determina el máximo real de bloques del dispositivo, y los bloques libres para archivos son:

$D = N \times 2048 \times 8 - 1$ (arranque) $- N$ (mapa bloques) $- 1$ (mapa descriptores) $- K$ (descriptores)

Bloque 4 — Asignación de espacio en disco

Conceptos clave

Encadenamiento (lista enlazada): cada bloque del archivo guarda, además de los datos, un **puntero al siguiente bloque**. Ventaja: no hay fragmentación externa. Desventaja: el **acceso aleatorio es lento** (para llegar al bloque n hay que recorrer los $n-1$ anteriores) y se pierden algunos bytes de cada bloque en el puntero.

Indexación: se reserva un **bloque de índices** que guarda los punteros a todos los bloques del archivo. Ventaja: acceso aleatorio directo y los bloques de datos quedan **enteros para datos**. Desventaja: el bloque de índices puede **desaprovechar espacio** si guarda pocos índices.

Ejercicio 14 — Encadenamiento vs. indexación

Enunciado. Disco de 30.000 KB, asignación por encadenamiento, bloque = 512 B. El archivo 'examen' ocupa los bloques $7 \rightarrow 12 \rightarrow 5 \rightarrow 3 \rightarrow 14$ (comienzo 7, fin 14: 5 bloques).

a) Tamaño máximo de datos del archivo (encadenamiento)

Paso 1 — Cuántos bloques hay que direccionar (para saber el tamaño del puntero):

| $N^\circ \text{ bloques del disco} = 30.000 \text{ KB} / 512 \text{ B} = 30.720.000 / 512 = 60.000 \text{ bloques}$

Paso 2 — Tamaño del puntero. Para direccionar 60.000 bloques hacen falta al menos 16 bits ($2^{16} = 65.536 \geq 60.000$), es decir **2 bytes**. Esos 2 bytes se restan a cada bloque:

| $\text{Datos útiles por bloque} = 512 \text{ B} - 2 \text{ B (puntero)} = 510 \text{ B}$

Paso 3 — El archivo usa 5 bloques:

| $\text{Tamaño máx.} = 510 \text{ B} \times 5 \text{ bloques} = 2.550 \text{ B}$

Respuesta: El tamaño máximo de datos del archivo 'examen' es **2.550 bytes**.

b) Problema, solución y pérdida de espacio

Problema: con encadenamiento el **acceso aleatorio es lento**, porque localizar un bloque obliga a recorrer todos los bloques intermedios de la cadena.

Solución: la **asignación por indexación**. El directorio apunta a un bloque de índices con los punteros a los bloques de datos; así **todo el bloque de datos queda disponible** para datos.

¿Varía el tamaño máximo? Sí, aumenta levemente porque ya no se gastan 2 bytes por bloque de datos:

| $\text{Tamaño con indexación} = 512 \text{ B} \times 5 = 2.560 \text{ bytes}$

¿Hay pérdida de espacio? Sí, en el **bloque de índices**: se le asigna un bloque entero (512 B) pero sólo guarda 5 índices de 2 B:

| $\text{Usado en el bloque de índices} = 5 \times 2 \text{ B} = 10 \text{ B}$

| $\text{Desaprovechado} = 512 \text{ B} - 10 \text{ B} = 502 \text{ B}$

Respuesta: Con indexación el máximo pasa a **2.560 B**; la pérdida de espacio es de **502 B** en el bloque de índices.

Bloque 5 — Planificación del brazo del disco

Conceptos clave

El **tiempo de búsqueda (seek)** es el que tarda el brazo en moverse entre cilindros y domina el tiempo de acceso. Los algoritmos de planificación deciden **en qué orden atender los pedidos** para minimizar el recorrido total del brazo (la suma de las distancias entre cilindros consecutivos).

- **FIFO (FCFS)**: atiende en el orden de llegada. Simple pero ineficiente.
- **SSTF**: atiende siempre el pedido **más cercano** a la posición actual. Riesgo de inanición de pedidos lejanos.
- **SCAN (ascensor)**: se mueve en una dirección hasta el **extremo del disco**, luego invierte. Atiende todo lo que encuentra al pasar.
- **LOOK**: como SCAN pero invierte en el **último pedido** de cada sentido, sin llegar al extremo físico.
- **C-SCAN**: sólo atiende en **un sentido**; al llegar al extremo **salta** al otro extremo (sin atender) y vuelve a recorrer. Da tiempos de espera más uniformes.
- **C-LOOK**: como C-SCAN pero el salto es del **último al primer pedido**, no a los extremos físicos.

En todos: **posición inicial = 46**, el brazo **se mueve hacia arriba** (cilindros mayores), y se considera el disco entre los cilindros **0 y 200**. Pedidos: 195, 50, 45, 12, 21, 12, 3, 8, 175, 15.

Ejercicio 15 — Cálculo del recorrido por cada algoritmo

FIFO — orden de llegada

```
46→195→50→45→12→21→12→3→8→175→15
149+145+5+33+9+9+9+5+167+160 = 691
```

Respuesta: Recorrido total = **691** cilindros.

SSTF — siempre el más cercano

```
46→45→50→21→15→12→12→8→3→175→195
1+5+29+6+3+0+4+5+172+20 = 245
```

Respuesta: Recorrido total = **245** cilindros.

SCAN — sube al extremo (200) y baja

Sube atendiendo 50, 175, 195 hasta el extremo 200; luego baja atendiendo 45, 21, 15, 12, 12, 8, 3.

```
46→200 (154) + 200→3 (197) = 351
```

Respuesta: Recorrido total = **351** cilindros (según el desarrollo de la clave).

Atención: La clave **imprime 723** como total de SCAN, pero su propia fila de tiempos suma **351** (4+125+20+5+155+24+6+3+0+4+5). El 723 es una errata. (Si se exige el SCAN estricto que además llega hasta el cilindro 0, el total sería 354.)

LOOK — invierte en el último pedido

Sube sólo hasta 195 (pedido más alto) y baja hasta 3 (pedido más bajo).

```
46→195 (149) + 195→3 (192) = 341
```

Respuesta: Recorrido total = **341** cilindros.

Atención: La clave imprime **346**; el valor correcto es **341** (la clave usó 155 en vez de 150 para el tramo 195→45).

C-SCAN — sube al extremo, salta a 0 y vuelve a subir

Sube atendiendo 50, 175, 195 hasta 200; salta a 0 (sin atender); sube atendiendo 3, 8, 12, 12, 15, 21, 45.

$$46 \rightarrow 200 \text{ (154)} + [\text{salto } 200 \rightarrow 0, \text{ no se cuenta}] + 0 \rightarrow 45 \text{ (45)} = 199$$

Respuesta: Recorrido total = **199** cilindros.

Atención: La clave imprime **191** para C-SCAN, pero ese valor corresponde a C-LOOK. El recorrido correcto de C-SCAN es **199** (8 más que C-LOOK porque sube hasta 200 en lugar de 195 y parte desde 0 en lugar de 3).

C-LOOK — del último al primer pedido

Sube hasta 195 (último pedido del sentido); salta a 3 (primer pedido, sin atender); sube atendiendo 3, 8, 12, 12, 15, 21, 45.

$$46 \rightarrow 195 \text{ (149)} + [\text{salto } 195 \rightarrow 3, \text{ no se cuenta}] + 3 \rightarrow 45 \text{ (42)} = 191$$

Respuesta: Recorrido total = **191** cilindros.

Resumen comparativo — Ejercicio 15

Algoritmo	Recorrido correcto	Total en la clave
FIFO	691	691 ✓
SSTF	245	245 ✓
SCAN	351	723 (errata)
LOOK	341	346 (errata)
C-SCAN	199	191 (errata)
C-LOOK	191	191 ✓

Para este conjunto de pedidos, los algoritmos circulares (C-LOOK 191, C-SCAN 199) y SSTF (245) son los más eficientes, mientras que FIFO (691) es el peor por atender los dos pedidos lejanos (195 y 175) intercalados.

Ejercicio 16 — Llega el pedido 47 mientras el brazo busca el 45

Enunciado. Igual que el ejercicio 15, pero mientras el brazo va en busca de la pista 45 se solicita la pista 47. ¿Qué pasa con cada algoritmo?

La clave de la situación es **cuándo** llega el 47: el brazo ya está bajando hacia el 45 (es de los últimos pedidos en atenderse). Según el algoritmo, el 47 cae sobre la trayectoria, se atiende en la próxima pasada, o va al final de la cola.

FIFO

El 47 se agrega **al final** de la cola (llega último). Se atiende después del 15.

```
| ... 175→15→47 → total = 691 + |15-47| = 691 + 32 = 723
```

Respuesta: Recorrido = **723** cilindros.

Atención: La clave imprime **591**, pero su fila de tiempos suma **723** (los mismos 691 del ej. 15 más 32 del tramo 15→47). El 591 es una errata.

SSTF

Cuando llega el 47, el brazo ya iba hacia el 45 (lo más cercano). Tras atender el 45, lo más cercano es el **47** (distancia 2) antes que el 50 (distancia 5). Se inserta el 47 entre 45 y 50.

```
| 46→45→47→50→21→15→12→12→8→3→175→195
| 1+2+3+29+6+3+0+4+5+172+20 = 245
```

Respuesta: Recorrido = **245** (no cambia: 45→47→50 cuesta lo mismo que 45→50 directo).

SCAN

El brazo ya subió hasta 200 y bajaba; el 47 se atiende en la **bajada**, justo antes del 45 (está sobre el camino), por lo que no agrega recorrido.

```
| 46→200 (154) + 200→47 (153) + 47→45 (2) + 45→3 = 351
```

Respuesta: Recorrido = **351** (igual que el ej. 15).

LOOK

Mismo razonamiento que SCAN: el 47 cae sobre la bajada, antes del 45.

```
| 46→195 (149) + 195→47 (148) + 47→45 (2) + 45→3 (42) = 341
```

Respuesta: Recorrido = **341** (igual que el ej. 15).

C-SCAN

Como sólo atiende **subiendo**, y el brazo ya pasó el 47 en la subida anterior, el 47 debe esperar a la **siguiente pasada ascendente** y se atiende **después** del 45 (45→47).

```
| 46→200 (154) + 0→45 (45) + 45→47 (2) = 201
```

Respuesta: Recorrido = **201** (2 más que en el ej. 15).

C-LOOK

Igual que C-SCAN: el 47 se atiende al final de la pasada ascendente, después del 45.

```
| 46→195 (149) + 3→45 (42) + 45→47 (2) = 193
```

Respuesta: Recorrido = **193** (2 más que en el ej. 15).

Resumen comparativo — Ejercicio 16

Algoritmo	Recorrido correcto	¿Cambió vs. ej. 15?
FIFO	723	Sí (+32): el 47 va al final de la cola
SSTF	245	No: el 47 cae entre 45 y 50
SCAN	351	No: el 47 cae sobre la bajada
LOOK	341	No: el 47 cae sobre la bajada
C-SCAN	201	Sí (+2): espera la próxima subida
C-LOOK	193	Sí (+2): espera la próxima subida

Conclusiones

Un pedido que llega '**a tiempo**' en la **dirección de avance** (como en SCAN y LOOK aquí) se atiende **sin costo extra**, porque cae sobre la trayectoria que el brazo iba a recorrer igual.

En los algoritmos **circulares** (C-SCAN, C-LOOK), un pedido que llega '**tarde**' (cuando el brazo ya pasó esa zona en el sentido único de atención) debe **esperar a la próxima pasada**, agregando un pequeño costo (+2 acá).

En **FIFO** el pedido nuevo siempre se atiende al final, lo que puede ser muy ineficiente. **SSTF** puede absorber el pedido sin costo si queda en una zona ya cercana. En conjunto, el ejercicio muestra que la **eficiencia de cada algoritmo depende del momento y la posición en que llegan los pedidos**, no sólo del conjunto inicial.

Fin de la Guía Práctica N° 7. ¡Éxitos en el examen!